

Week 4

1. The UNIX File I/O System (cont'd)

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Monday, September 22nd, 2025



University of Windsor

Content

- 1 C: Parameters and Arguments
- 2 C: Pointers
- 3 Binary I/O
- 4 Random file access
- 5 Formatted I/O
- 6 The `errno` variable and the function `perror()`
- 7 Summary

Part I

C: Parameters and Arguments in Functions



C: Functions

Function Prototype

return type function name parameters (arguments)

```
HEADER { int heading ( void )
```

NO semicolon

```
{  
  BODY { //statements  
        return 0;  
    }  
}
```



Reference: <https://www.geeksforgeeks.org/c/c-function-argument-return-values/>

C: Parameters

Parameters and Arguments

Information can be passed to functions as a parameter. Parameters act as variables inside the function.

Parameters are specified after the function name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma:

Syntax

```
returnType functionName(parameter1, parameter2, parameter3) {  
    // code to be executed  
}
```

Reference: https://www.w3schools.com/c/c_functions_parameters.php



C: Parameters

In the example below, the function takes a string of characters with **name** as parameter. When the function is called, we pass along a name, which is used inside the function to print "Hello" and the name of each person:

Example

```
void myFunction(char name[]) {  
    printf("Hello %s\n", name);  
}  
  
int main() {  
    myFunction("Liam");  
    myFunction("Jenny");  
    myFunction("Anja");  
    return 0;  
}
```

Reference: https://www.w3schools.com/c/c_functions_parameters.php



Part II

C: Pointers



C Pointers

[< Previous](#)[Next >](#)

Creating Pointers

You learned from the previous chapter, that we can get the **memory address** of a variable with the reference operator `&`:

Example

```
int myAge = 43; // an int variable

printf("%d", myAge); // Outputs the value of myAge (43)
printf("%p", &myAge); // Outputs the memory address of myAge (0x7ffe5367e044)
```

[Try it Yourself »](#)

A **pointer** is a variable that **stores** the **memory address** of another variable as its value.

A **pointer variable points** to a **data type** (like `int`) of the same type, and is created with the `*` operator.

The address of the variable you are working with is assigned to the pointer:

Reference: https://www.w3schools.com/c/c_pointers.php



Pointers in C

Last Updated : 18 Sep, 2025



A pointer is a variable that stores the memory address of another variable. Instead of holding a direct value, it holds the address where the value is stored in memory. It is the backbone of low-level memory manipulation in C. Accessing the pointer directly will just give us the address that is stored in the pointer. For example,

```
#include <stdio.h>

int main() {

    // Normal Variable
    int var = 10;

    // Pointer Variable ptr that
    // stores address of var
    int* ptr = &var;

    // Directly accessing ptr will
    // give us an address
    printf("%d", ptr);

    return 0;
}
```

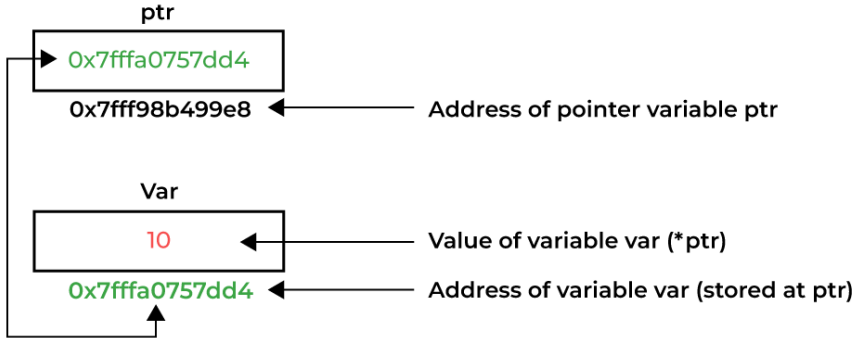
Reference: <https://www.geeksforgeeks.org/c/c-pointers/>



Output

0x7ffa0757dd4

This hexadecimal integer (starting with 0x) is the memory address.



Reference: https://www.w3schools.com/c/c_pointers.php



Part III

Binary I/O



Binary I/O

The following I/O functions **are not well suited** for binary I/O:

`fgetc()` / `fputc()` are too slow to handle millions of bytes.

`fgets()` cannot be used because it stops when it hits a newline character which is usually part of the binary information.

`fputs()` cannot be used because it stops when it hits a `NULL` character which is usually part of the binary information.

This makes the above I/O functions **not well suited** for binary I/O.

Binary I/O functions

- Binary I/O functions are especially useful when reading/writing whole structures.
- Instead of reading/writing fields, we can read/write a number of bytes (the size of the structure) and therefore, achieve fast input/output operations.

fread

The read function in the `stdio`

```
size_t fread(void *ptr, size_t size, size_t  
nitems, FILE *fp);
```

In particular, `fread()`

- Reads into an array pointed to by `ptr` up to `nitems` items of data from the stream `fp`, where an item of data is a sequence of `size_t` bytes.
- Stops reading bytes if an end-of-file or error condition occurs, or if `nitems` items have been read.
- Returns the number of items (not bytes) read.

When the returned value is less than `nitems`, `feof()` or `ferror()` should be called to verify the reason.

fwrite

The write function in the `stdio`

```
size_t fwrite(const void *ptr, size_t size,  
size_t nitems, FILE *fp);
```

In particular, `fwrite()`

- Writes to the stream `fp` up to `nitems` items of data from the array, pointed to by `ptr`, where an item of data is a sequence of `size_t` bytes.
- Stops writing bytes when it has written `nitems` items of data or if an error has occurred.
- Returns the number of items (not bytes) read.

An important issue about `fwrite()`

A call to `fwrite()` in buffered mode may return success even though the underlying system call (`write()`) fails. This can cause unpredictable results. **Therefore, use `fwrite()` function only in unbuffered mode if needed.**

Example: speeding-up input using `fread()`

```
typedef struct{
    char name[30];
    int id;
    double x, y;
} itemT;

int main(void){
    itemT data[100];
    FILE *fp;

    fp = fopen("data.bin", "r");
    if(fread(data, sizeof(itemT), 100, fp)!=100){
        if(feof(fp))
            printf("End of file before reading all");
        else
            printf("Error occurred");
        exit(1);
    }
}
```

Example: speeding-up output using `fwrite()`

```
int main(void){
    itemT data[100];
    FILE *fp;

    :

    fp = fopen("result.bin", "w");
    setbuf(fp, NULL); // turn buffering off
    if(fwrite(data, sizeof(itemT), 100, fp) != 100){
        printf("Error occurred");
        exit(1);
    }
}
```


Homework → See `pirate.c` and `dictionary.c` for two more examples.

Problems with binary I/O (Computer Architecture-wise)

There are two fundamental problems with these binary I/O functions:

- 1 The offset of a structure's member can differ from one compiler to another.
→ The binary layout of a structure can differ depending on compiler options.
- 2 The binary format used to store a multibyte numeric value differs from one architecture to another, known as *little-endian* vs. *big-endian* orders.

For example:

Sparc workstations are big-endian : a 4-byte integer is stored with the left-most byte first.

Intel machines are little-endian : a 4-byte integer is stored with the right-most byte first.

Case example

1-Run this on CS server to create the file `data.bin`.

```
#include <stdio.h>
```

0x 12 34

```
#include <stdlib.h>
```

```
int main(int argc, char *argv[]){
```

```
    FILE *f1;
```

```
    int data[10], i;
```

Addr: value

1000: 12

```
    for(i=0; i<10; i++){
```

1001: 34

```
        data[i] = 10*i*i;
```

```
    if(!(f1=fopen(argv[1], "w"))){
```

```
        printf("could not create file");
```

```
        exit(1);
```

Addr: value

1000: 34

```
    }
```

1001: 12

```
    if(fwrite(data, sizeof(int), 10, f1) != 10){
```

```
        printf("Error on writing into file");
```

```
        exit(2);
```

```
    }
```

```
    fclose(f1);
```

```
}
```

- 2 Run this on CS server with `data.bin` as `argv[1]`
- 3 ftp `data.bin` to a PC machine
- 4 Run this on a PC with `data.bin` as `argv[1]`

```
#include <stdio.h>

#include <stdlib.h>

int main(int argc, char *argv[]){
    FILE *f1;
    int data[10], i;

    if(!(f1=fopen(argv[1], "r"))){
        printf("could open file");
        exit(1);
    }
    if(fread(data, sizeof(int), 10, f1) != 10){
        printf("Could not read data");
        exit(2);
    }
    for(i=0; i<10; i++){
        printf("%d\n", data[i]);
    }
}
```

Part IV

Random File Access



Random file access: `ftell`, `rewind` and `fseek`

The standard I/O library provides functions to

get the current value of the file-position indicator of a stream.

```
long ftell(FILE *stream);
```

If successful, it returns the current value of the file-position indicator, measured in bytes from the beginning of the file, -1 otherwise.

reset file position indicator in a stream

```
void rewind(FILE *stream);
```

Sets the value of the file-position indicator to 0.

`rewind()` also clears the error/end-of-file indicators.

redefine a file-position indicator in a stream

```
int fseek(FILE *fp, long offset, int  
whence);
```

Return 0 if OK, nonzero otherwise.

`fseek()`

```
int fseek(FILE *fp, long offset, int whence);
```

`fseek()` redefines the position of the next input or output operation. The new position, measured in bytes from the beginning of the file, is obtained by adding `offset` to the position specified by `whence`, whose values are defined in `<stdio.h>` as follows:

`SEEK_SET`: set position equal to `offset` bytes.

`SEEK_CUR`: set position to current location plus `offset`.

`SEEK_END`: set position to `EOF` plus `offset`.

In particular, `fseek(fp, 0L, SEEK_SET)` is equivalent to `rewind(fp)`

Note

Note that the `fseek()` function allows the file-position indicator to be set beyond the end of existing data in the file. Reads of data in the gap will return bytes with the value 0.
These functions can only be used on random access files.

Part V

Formatted I/O



Formatted output

The standard I/O library provides the following functions:

```
int printf(const char *format, ...);  
int fprintf(FILE *fp, const char *format,  
...);  
int sprintf(char *ar, const char *format,  
...);
```

sprintf is similar to `printf`, except that it writes to the array `ar` instead of the standard output and it adds a `NULL` character at the end.

All these function return the number of written characters (excluding the terminating `NULL` character in the case of `sprintf`) and a negative value in case of error.

Example: `sprintf(result, ``%f``, speed);`
This converts a `float(speed)` into a string (`result`).

Formatted input

The standard I/O library provides the following functions:

```
int scanf(const char *format, ...);
int fscanf(FILE *fp, const char *format,
...);
int sscanf(const char *ar, const char
*format, ...);
```

sscanf is similar to `scanf`, except that it reads inputs from the array pointed to by `ar` instead of the standard input `stdin`.

All these functions return the number of items successfully read and -1 in case of end-of-file or error.

Example

Example1: `sscanf(argv[1], ``%d``, &n);`

Converts the first command argument `argv[1]`, which is read as a string, into an integer and stores it in the variable `n`.

Homework → Example2: Run-time-error-free input functions (see `readInt.c`)

```
int readInt(void) {
    char input[100];
    int value;

    do{
        printf("Please enter an integer> ");
        scanf("%s", input); //17.6
    }while (sscanf(input, "%d", &value) != 1);
    return(value);
}

//DIY
double readDouble(void) {
    char input[100];
    double value;

    do{
        printf("Please enter a real number> ");
        scanf("%s", input);
    }while (sscanf(input, "%lf", &value) != 1);
    return(value);
}
```

Part VI

errno and perror ()



`errno` and `perror()`

All function calls may fail when an error occurs and returns -1 as a consequence.

The standard I/O library functions will fail whenever a related system call fails.

E.g., `fopen()` depends on the system call `open()`.

→ `fopen()` will return a `NULL` if `open()` fails.

To gain more information about system call failures, Unix provide two things, a globally accessible integer variable, called `errno`, and a subroutine, called `perror()`.

In order to use these two tools, you should include `<sys/errno.h>` in your program.

More details: `errno` and `perror()`

`errno`: Every process contains a global variable, called, `errno`, initialized to 0 when the process is first created.

When a system call error occurs, `errno` is set to a numerical code corresponding to the error.

For example, if `open()` fails because the file does not exist, then `errno` is set to 2.

On the other hand, if a system call was successful, **`errno` will not be updated.**

Note that the list of error codes can be found in `<usr/include/errno.h>`.

More details: `errno` and `perror()`

```
void perror(char *str):
```

This library routine is provided by Unix for reporting errors.

When called, it will produce a message on the standard error that consists of the string parameter passed to the routine, a colon (:) and, an additional english message associated with the current value of the variable `errno`.

```
#include <stdio.h>

void copyFiles(FILE *, FILE *);

int main(int argc, char *argv[]){
    FILE *f1, *f2;

    f1=fopen(argv[1], "r");
    if( f1 == NULL){
        perror("main function");
        exit(1);
    }
    f2=fopen(argv[2], "w");
    if( f2 == NULL){
        perror("main function");
        exit(2);
    }
    copyFiles(f1, f2);
    exit(0);
}
```



```
void copyFiles(FILE *f1, FILE *f2){
    int c;

    while((c=fgetc(f1)) != EOF)
        if (fputc(c, f2) == EOF){
            perror("copy function");
            exit(3);
        }

    if(ferror(f1)){
        perror("copy function");
        exit(4);
    }
}
```

Note that `perror()` is not a system call.

Part VII

Homework



Content

```
hello.c
```

```
xxd example.txt  
filesize1.c
```

Content

char and int comparison			char to int promotion		
Decimal	int	char	char	unsigned	signed
2	00 02	02	02	00 02	00 02
1	00 01	01	01	00 01	00 01
0	00 00	00	00	00 00	00 00
-1	FF FF	FF	FF	00 FF	FF FF
-2	FF FE	FE	FE	00 FE	FF FE

Content

```
char c;  
  
while ((c = fgetc(fp)) != EOF)  
{  
    putchar (c);  
}
```

char_int.c

assume that within the file we are reading from is a byte with value 0xff.

`fgetc()` returns this value within an int, so it looks like this: **0x00ff** (again, assuming 2 byte integers).

To store this value in a char, it must be **demoted**, and the char value becomes **0xff**. Next, the char `c` is compared with the int `EOF`.

Promotion rules apply, and `c` must be promoted to an int. However, in the sample code, the sign of `c` isn't explicitly declared, so we don't know if it's signed or unsigned, so the int value could become either 0xff ff or 0x00 ff.

Therefore, the code is not guaranteed to work in the way we require.

Content

```
unsigned char c;

while ((c = fgetc(fp))
!= EOF) {
    putchar
(c);
}
```

Another scenario to consider is where the char is **unsigned**.

In this case, the process of demoting and promoting the returned value from `fgetc()` will have the effect of “corrupting” the EOF value, and the program will get stuck in **an infinite loop**.

Content

```
unsigned char c;

while ((c = fgetc(fp)) !=
EOF) {
    putchar (c);
}
```

- EOF (**0xff ff**) is returned by `fgetc()` due to end of input
- Value demoted to **0xff** to be stored in unsigned char `c`
- unsigned char `c` promoted to an int, value goes from **0xff** to **0x00 ff**
- EOF is compared with `c`, meaning comparison is between **0xff ff** and **0x00 ff**.
- The result is FALSE (the values are different), which is undesirable.
- `fgetc()` is called again, and still returns EOF. The endless loop begins.

Part VIII

Summary & Quiz



We have looked at almost all functions provided by the standard I/O library.

This library is used by most Unix applications.

In particular, you have to be aware of the following:

- Calling a standard I/O function does not necessarily trigger an actual I/O operation.
- Buffering generates confusion and problems.
- I/O are main sources of serious bugs
- It is very important to understand each I/O function before using it to get a robust software.

Quiz



Attendance Quiz

701968234901768023405879312016
670980321768767



774862586034187069210783468019
701968234901768023405879312016
670980321768767

WPOYFXI LFJXI G I E
H I MPHLOM ZFOLI PG
BXNYSXWHYGI JLG I L
OLI EPC OOUJPL QDZC
NYDWIGLIPJL QDZC

Thank you!
Questions?

DEFYXHQGRUOSGRP RN	3236343972474116
QJRMVQJXVRI DKUEVP	893833239594578
KGWBGQFHP RSUP RU	0310019701552750
XURNYUXZ DCSYUUM I	7875465400196596
WHMFSP UIQYSUQYRSHRG	5019353606937419

00110011100111
11111010000100
11000000001111
00111100111100
00101111001100
00000111011011
11001100110011

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor